

SURROUND SOUND V2

Real-time environmental, sound-event,
and speech recognition from
microphone audio

Everett-Alan Hood



OBJECTIVES



Classify a user's environment in real time

This work is heavily inspired by the **DCASE** (Detection and Classification of Acoustic Scenes and Events) Challenge Community.

While their work separates classification into distinct categories, this extension attempts to combine them into a **cohesive scene analyzer**.



Detect multiple audible situations simultaneously

The idea is to use **multiple models** that specialize in **different aspects** of a scenario. Then, summarize the scene as a whole using a large language model. This project separates **environment**, **events**, and **speech**. *Speech is now a part of the pipeline!*



Summarize the current auditory scenario

Just for a more understandable interpretation of the current scenario, **GPT 4.1 Mini** was used to quickly summarize the scenario based on the models' findings.

This is not counted in the final evaluation, it's simply a quality of life enhancement.



Implement modern deep learning techniques

One of the primary goals was to improve the pipeline's performance. Modern ASR, ASC, and DL features were added to increase those metrics. These include **channel attention**, **residual skip connections**, **random search**, **BCE & focal loss**, **increased datasets**, and more.

DATASET & PREPROCESSING

Environment

Google AudioSet

2M+ annotated video clips via YouTube

- Downloaded 100K+ VGGish embeddings instead of raw WAV audio
- ~15K+ Balanced, 88K+ Unbalanced
- 16kHz mono, log-mel spectrograms
- normalized, max 10s clips
- extracted labels related to eight different categories
 - animal, crowd, indoor, other_ambient, outdoor_nature, vehicle_ground, vehicle_water, water

```
ALLOWED_LABELS = {  
    # Scenes / locations  
    "Outside, rural or natural", "Outside, urban or manmade", "Crowd",  
    "Inside, small room", "Inside, large room or hall", "Inside, public space",  
    "Field recording",  
  
    # Weather / geophony  
    "Wind", "Rain", "Rain on surface", "Thunder", "Thunderstorm",  
    "Waves, surf", "Ocean", "Stream", "Waterfall", "Raindrop", "Rustling leaves",  
    "Water",
```

Events

FSD50K

51K annotated audio clips via Freesound

- 208 Classes based on AudioSet
- 16kHz mono, log-mel spectrograms
- normalized, trimmed from 30s clips to 10s
- some labels had far more samples than others

```
PARENT_MAP = {  
    # Human sounds  
    "Speech": "Human sounds",  
    "Male speech, man speaking": "Human sounds",  
    "Female speech, woman speaking": "Human sounds",  
    "Child speech, kid speaking": "Human sounds",  
    "Conversation": "Human sounds",  
    "Narration, monologue": "Human sounds",  
    "Babbling": "Human sounds",  
    "Whispering": "Human sounds",  
    "Laughter": "Human sounds",  
    "Baby laughter": "Human sounds",  
    "Giggle": "Human sounds",  
    "Snicker": "Human sounds",  
    "Belly laugh": "Human sounds",
```

DATASET & PREPROCESSING

Speech

CHiME-6 + Common Voice

42K total clips

VAD (Voice Activation Detector)

2D CNN

CHiME-6 (OpenSLR 150)

- Far-field multi-speaker home recordings
- 17K segmented clips (10s windows)

Common Voice 25.0 (Mozilla)

- 25K single-speaker read speech clips
- Only 447 no-sound speech clips

Whisper large-v3 (Transcription)

- Best performing open-weight ASR model publicly available
- 680K hours of multilingual audio

```
search_results = []
for trial in range(N_TRIALS):
    hparams = sample_hparams()
    print(f"\n{'='*60}\nTRIAL {trial+1}/{N_TRIALS} hparams={hparams}\n{'-'*60}")
    best_f1, ckpt_path = run_trial(train_df, val_df, hparams, device, trial)
    search_results.append({"trial": trial+1, "best_macro_f1": best_f1,
                          "checkpoint": ckpt_path, **hparams})
    print(f" Trial {trial+1} best macro_f1: {best_f1:.4f}")

results_df = pd.DataFrame(search_results).sort_values("best_macro_f1", ascending=False)
results_df.to_csv(OUTPUT_DIR / "hparam_search_results.csv", index=False)

best_row = results_df.iloc[0]
best_ckpt = best_row["checkpoint"]
print(f"\n{'='*60}")
print(f"[INFO] Best trial: {int(best_row['trial'])} macro_f1={best_row['best_macro_f1']:.4f}")
print(results_df[["trial", "best_macro_f1", "learning_rate", "batch_size",
                  "dropout", "focal_gamma", "se_reduction", "base_channels"]].to_string(index=False))

final_path = OUTPUT_DIR / "best_speech_model.pt"
shutil.copy(best_ckpt, final_path)
print(f"[INFO] Best model -> {final_path}")

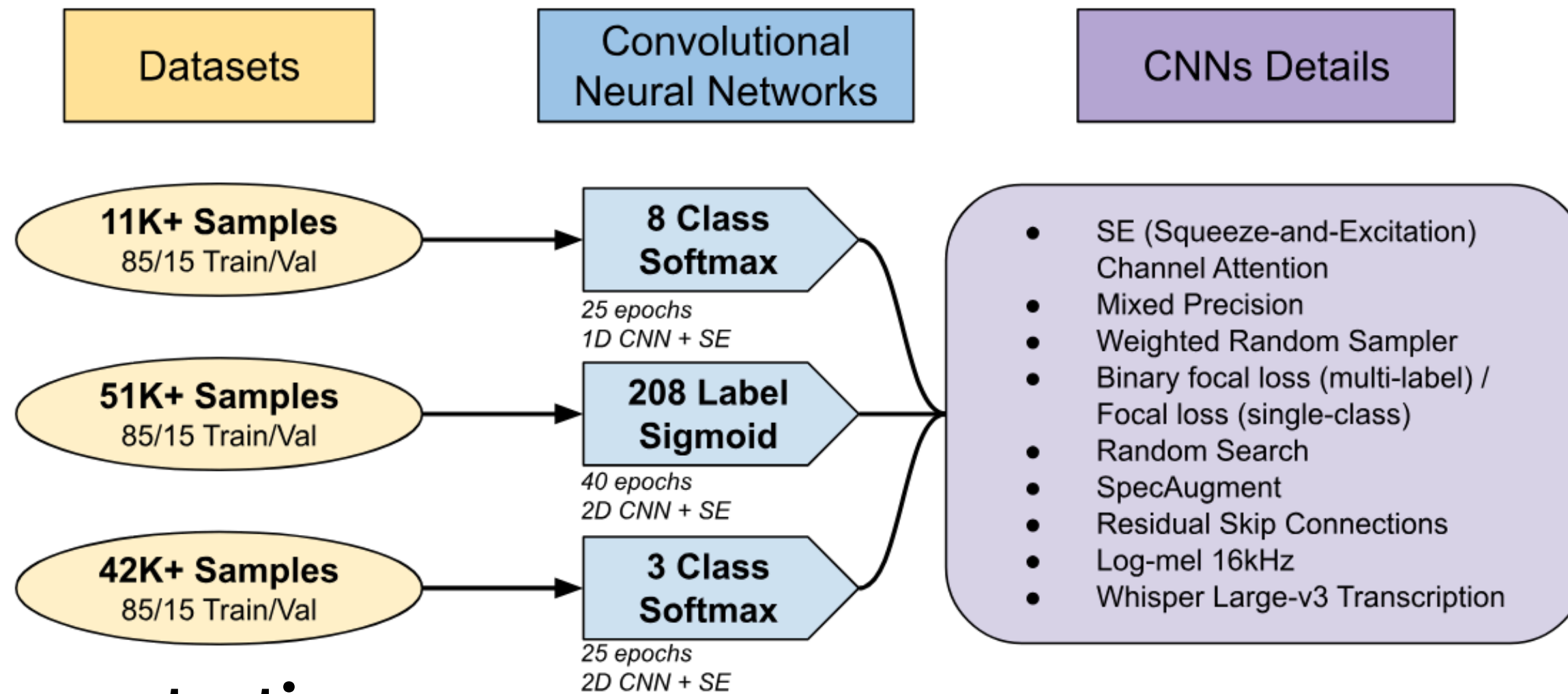
# Final report
ckpt = torch.load(final_path, map_location=device)
model = SpeechCNN(
    base_channels=int(best_row["base_channels"]),
    dropout=float(best_row["dropout"]),
    se_reduction=int(best_row["se_reduction"]),
).to(device)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()

val_ds = SpeechDataset(val_df, augment=False)
val_loader = DataLoader(val_ds, batch_size=128, shuffle=False,
                        num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY)
```

METHODOLOGY

Here's a quick overview about how these models are trained and how the live implementation works.

Models



Demonstration



LIVE DEMO?

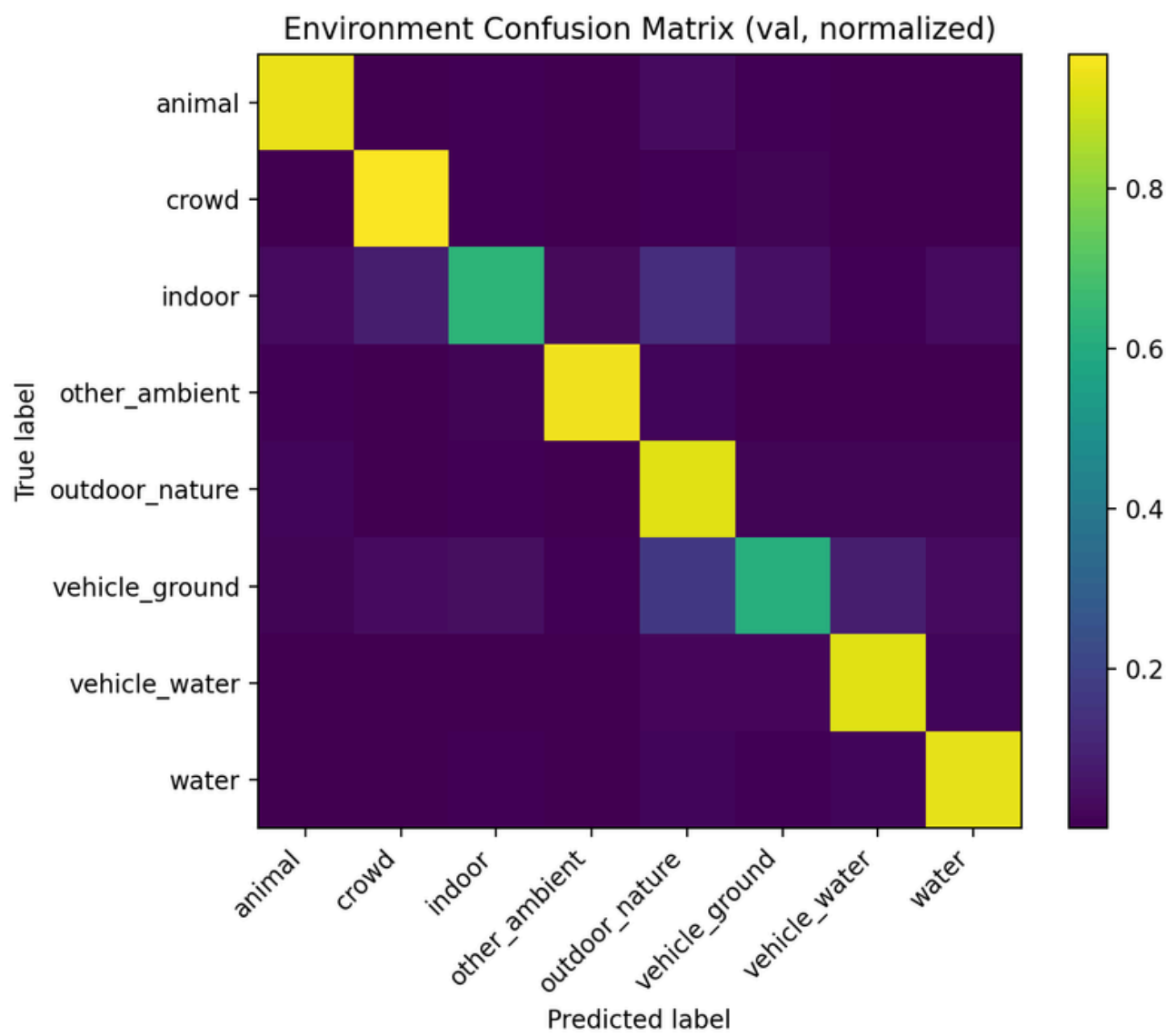
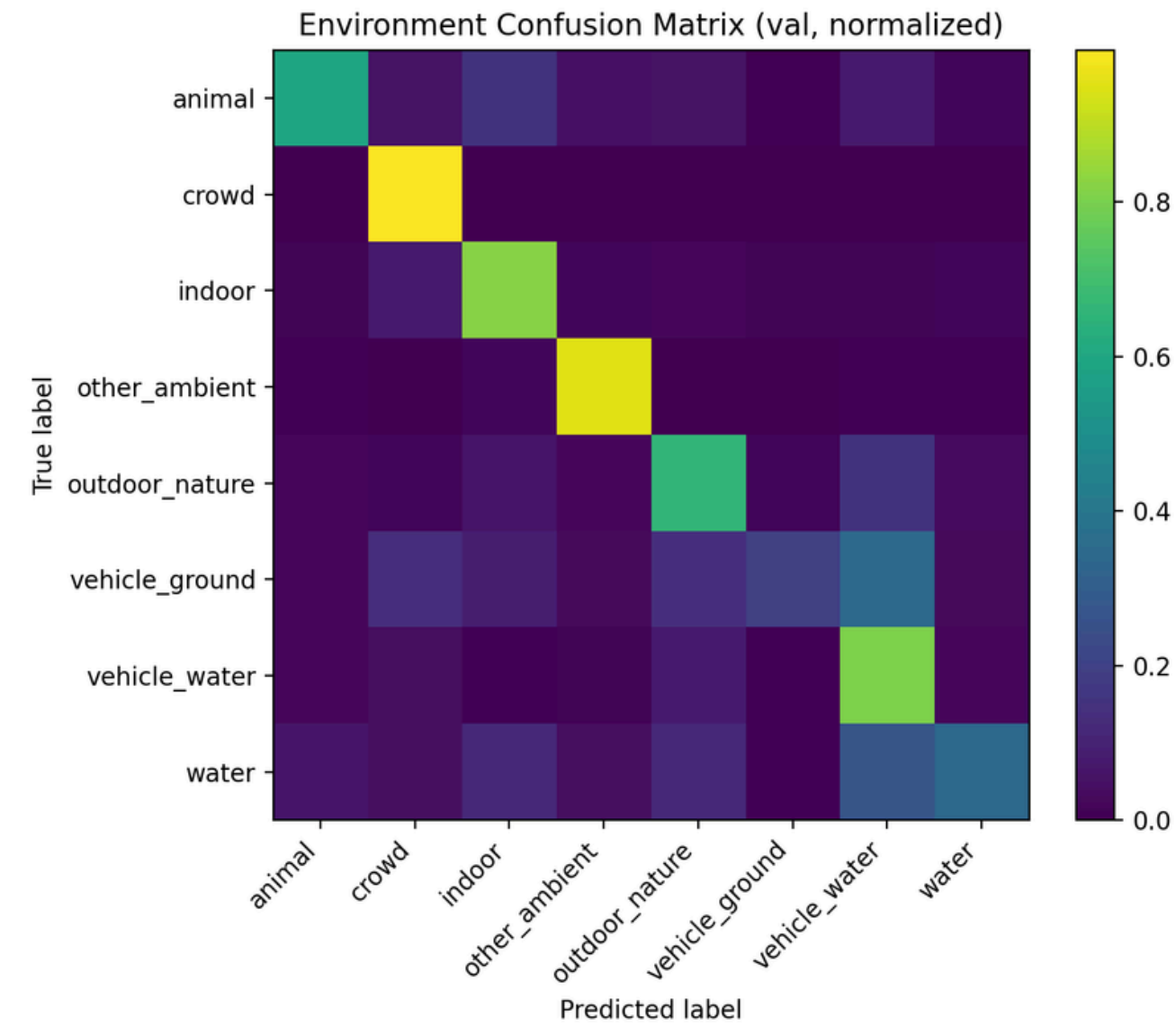


With your help, let's see if it can predict the current environment!

RESULTS - ENVIRONMENT

Average	P	R	F1
Macro V1	0.5404	0.6376	0.4985
Weighted V1	0.6818	0.4666	0.4466

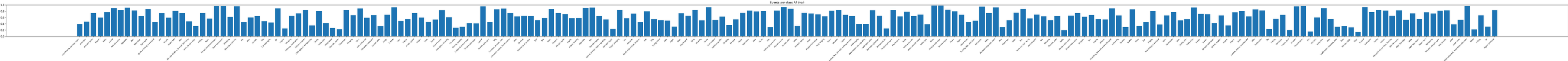
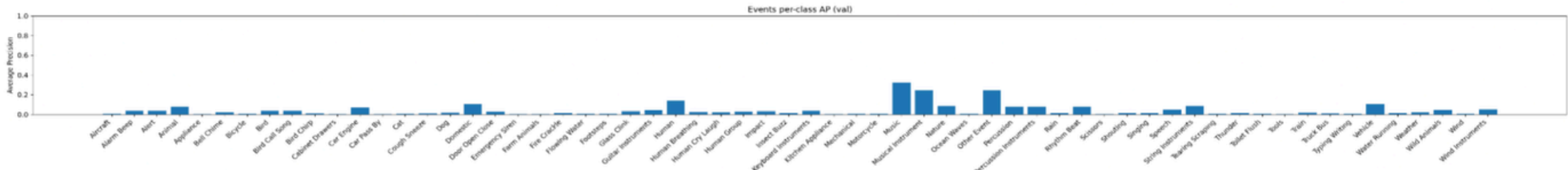
Average	P	R	F1
Macro V2	0.7104	0.8620	0.7537
Weighted V2	0.8113	0.7416	0.7457



RESULTS - EVENTS

Average	P	R	F1
---------	---	---	----

Macro V1	0.33	0.62	0.39
Macro V2	0.64	0.72	0.55



BEST LABELS

Class	# Samples	P	R	F1
Wind/Woodwind instrument	2653	0.92	0.90	0.91
Brass instrument	1035	0.86	0.94	0.90
Bowed string instrument	2064	0.87	0.91	0.89

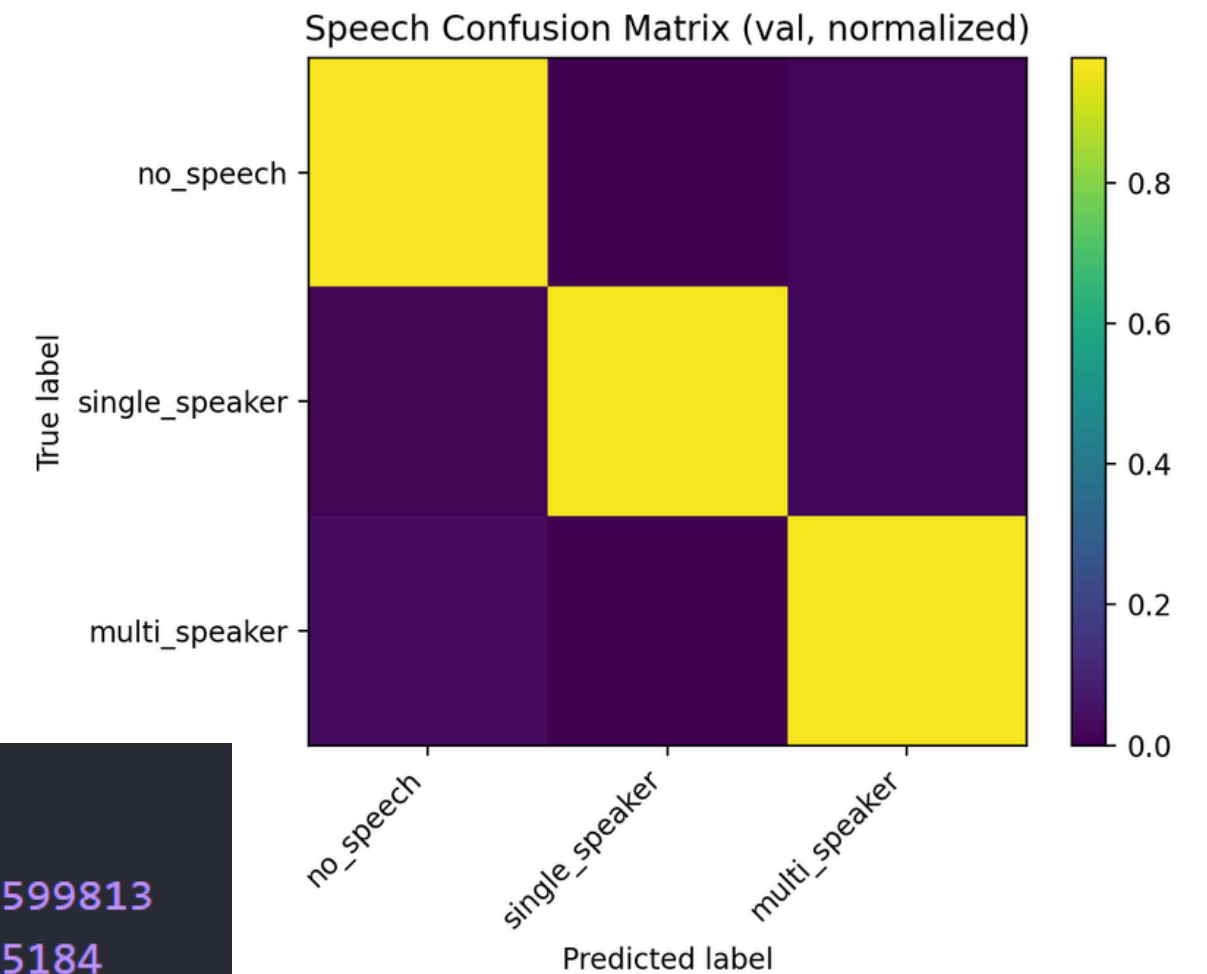
WORST LABELS

Class	# Samples	P	R	F1
Fill (with liquid)	148	0.13	0.93	0.22
Truck	154	0.11	0.79	0.21
Tick	109	0.11	0.81	0.19

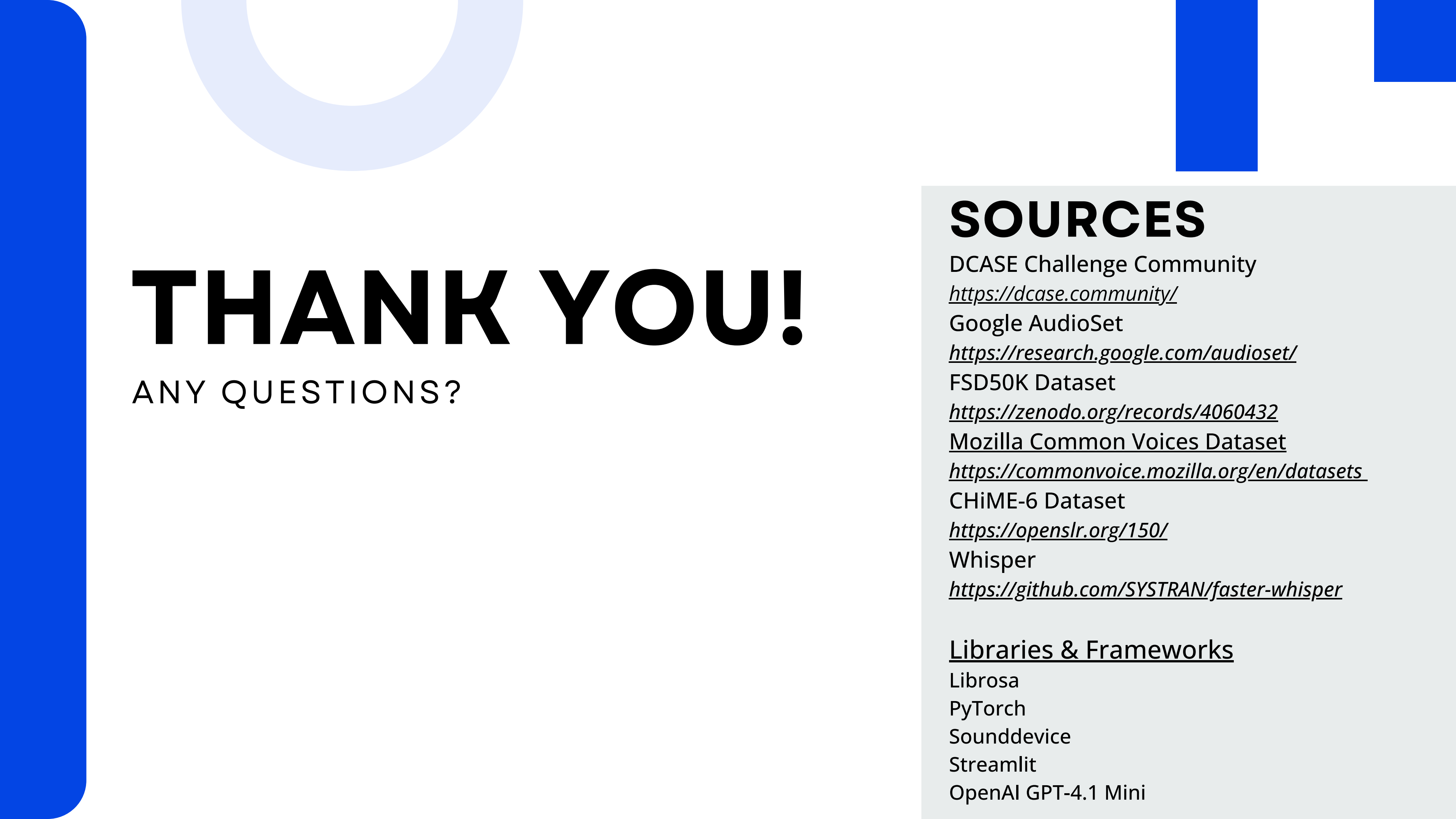
RESULTS - SPEECH

Class	# Samples	P	R	F1	A
No Speech	447	0.321	0.978	0.483	0.977
Single Speaker	25718	0.999	0.966	0.982	0.966
Multi Speaker	16778	0.968	0.963	0.965	0.963

```
class_id,class_name,precision,recall,f1,support,accuracy
0,no_speech,0.3208516886930984,0.9776286353467561,0.48313985627418465,447,0.9776286353467561
1,single_speaker,0.9989549419188875,0.9663659693599813,0.9823902602921122,25718,0.9663659693599813
2,multi_speaker,0.9677284157585918,0.9633448563595184,0.96531660692951,16778,0.9633448563595184
```



- 96.5% accuracy: strongest model in the pipeline
- single_speaker (F1=0.98) and multi_speaker (F1=0.97) are cleanly separated
- no_speech precision is low (0.32) due to only 447 training clips vs 42K+ speech
- in live conditions, brief or intermittent overlap can blur the single/multi boundary
- trained on clean read speech vs sustained overlap; real conversations fall in between
- did not extract the full dataset for Mozilla's Common Voices dataset



THANK YOU!

ANY QUESTIONS?

SOURCES

DCASE Challenge Community

<https://dcase.community/>

Google AudioSet

<https://research.google.com/audioset/>

FSD50K Dataset

<https://zenodo.org/records/4060432>

Mozilla Common Voices Dataset

<https://commonvoice.mozilla.org/en/datasets>

CHiME-6 Dataset

<https://openslr.org/150/>

Whisper

<https://github.com/SYSTRAN/faster-whisper>

Libraries & Frameworks

Librosa

PyTorch

Sounddevice

Streamlit

OpenAI GPT-4.1 Mini